UNIT - 2 (DAA) 22BCON1068

Tutorial sheet - 3

PAGE NO.: 11
DATE: 20/09/24

- Section - A

1. Difference between Greedy Technique and Dynamic programming?

Ans Greedy Approach: it make the choice at each step with the hope of finding a global optimal solution. It selects locally optimal solution at each stage without considering the overall effect on the solution.

Dynamic Programming: it breaks down a problem into smaller sub problems and solve each problem only once, storing its solution.

2. Differentiate between Backtracking and Branch and Bound Technique?

Ans Backtracking is the modified Procedure of the brute force Approach where the technique efficiently searches for a solution to the problem among all available options.

Branch and Bound technique is an algorithm used to solve the problem of optimization by breaking it down smaller sub-problems and by bounding function it eliminates the subproblem that doesn't contain an optimal solution.

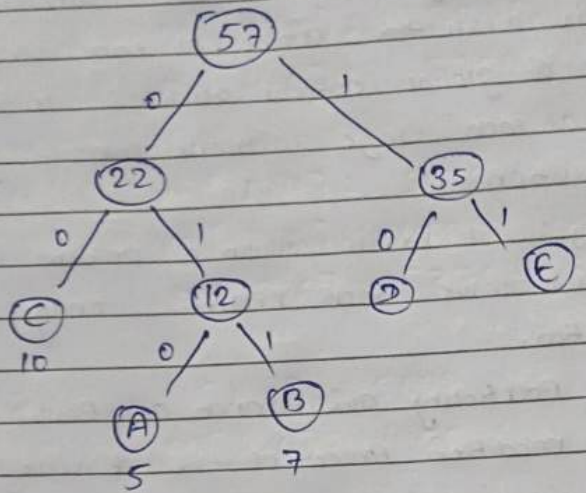
3. Explain the key concept behind Huffman coding briefly and illustrate the construction of a Huffman tree for the following character frequencies:

Character	Frequency
A	5
B	7
C	10
D	15
E	20

Ans Huffman coding is a lossless data compression algorithm. The idea is to assign variable length codes to input characters, lengths of the assigned codes are based on the frequencies of corresponding characters.

Oct 6, 2024, 22:47

Huffman tree for given character and frequencies:-



A = 010
B = 011
C = 00
D = 10
E = 11

4. What is Dynamic Programming? How this approach is different from recursion?

Ans Dynamic programming: it breaks down a problem into smaller subproblems and solve each subproblem only once, storing its solution.
Recursion does not store any value until reach to the final ~~storage~~ stage (base case) and Dynmic Programming is mainly an optimization compared to simple recursion.

5. Calculate how many ways we can multiply matrics if we are given n matrics. if you have been provided with 4 matrics, how many ways can you solve it?

Ans Total no. of way to we can we multiply matrics if we have n matrics →

$$C(n) = \frac{(2n)!}{(n! * (n+1)!)}$$

n=4 Calculate $C(n-1) = C_3$

$$C_3 = \frac{6!}{3!4!} = 5 \text{ ways}.$$

- Section - B (3×7 = 21)

1. Using dynamic programming find the length of the Longest Common Subsequence (LCS) between the following two strings:
String 1: 'AGGTAB'
String 2: 'GXTXAYB'

Sol

		G	X	T	X	A	Y	B
	0	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1	1
G	0	1	1	1	1	1	1	1
G	0	1	1	1	1	1	1	1
T	0	1	1	2	2	2	2	2
A	0	1	1	2	2	3	3	3
B	0	1	1	2	2	3	3	4

Length of the LCS is 4
The longest common subsequence is GTAB.

2. Given the matrices, A(10×30), B(30×5), C(5×60), D(60×10), E(10×5), and F(5×5), determine and illustrate the optimal parenthesization to minimize multiplications. Provide a step-by-step numerical solution and justify your approach.

Ans Matrix Chain multiplication for matrices.

A, B, C, D, E, F

A	B	C	D	E	F
(10×30)	(30×5)	(5×60)	(60×10)	(10×5)	(5×5)

L2:

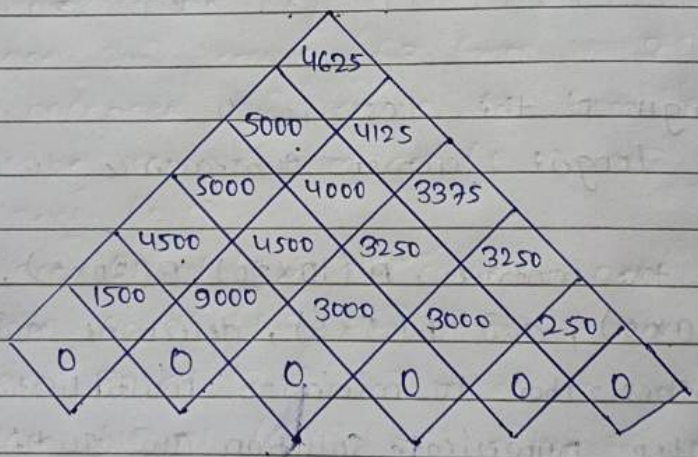
M[1,2] = 10×30×5 = 1500
M[2,3] = 30×5×60 = 9000
M[3,4] = 5×60×10 = 3000
M[4,5] = 60×10×5 = 3000
M[5,6] = 10×5×5 = 250

L3 = M[1,3] = 4500
M[2,4] = 4500
M[3,5] = 3250
m[4,6] = 3250

L4: m[1,4] = 5000
m[2,5] = 4000
m[3,6] = 3375

L5: M[1,5] = 5000
m[2,6] = 4125

L6: M[1,6] = 4625



3. Apply Backtracking to solve the 4-Queen Problem on a chessboard. Provide the Queen Placements ensuring no mutual threats and briefly explain your solution.

Ans The four Queens Problem consists in placing four Queens on a 4x4 chessboard so that no two Queens attack each other. That is no two queens are allocated to be placed on the same row, the same column, on the same diagonal.

22BCON1068

PAGE NO.: 15
DATE: / /

Solution for n=4 on a 4x4 chessboard.

Using Backtracking Algorithm:
Place each queen one by one in different rows, starting from the topmost row. While placing a queen in a row, check for clashes with already placed queens. For any column, if there is no clash then mark this row and column as part of the solution by placing the queen. In case, if no safe cell found due to clashes then backtrack (i.e; undo the placement of recent queen) and return false.

Solution:
Step 0: Initialize a 4x4 board.
Step 1: Put our first Queen (Q_1) in the (0,0) cell.
. 'x' represent the cell which is not safe that they are under attack by the Queen (Q_1).
• After this move to the next row [0 → 1]

	0	1	2	3
0	Q_1	x	x	x
1	x	x		
2	x		x	
3	x			x

4x4

Step 2: Put on next queen (Q_2) in the (1,2) cell.
After this move to the next row.

	0	1	2	3
0	Q_1	x	x	x
1	x	x	Q_2	x
2	x	x	x	x
3	x		x	x

Oct 6, 2024, 22:47

Step 3: At row 2 there is no cell which are safe to place Queen (Q_3)
- So backtrace and remove Q_2 from cell (1,2)

Step 4: There is still a safe cell in the row 1. i.e. cell (1,3)
Put Queen (Q_2) at cell (1,3).

	0	1	2	3
0	Q_1	x	x	x
1	x	x	x	Q_2
2	x		x	x
3	x	x		x

Step 5: Put (Q_3) at cell (2,1).
Step 6: There is no any cell to place Q_4 at row 3.
- Backtrace and remove Q_3 from row 2
- Again backtrack and remove Q_2 from row 1
- Q_1 will be remove from cell (0,0) and move to next (0,1).

Step 7: Place Q_1 at (0,1) and move to next row.
Step 8: Place Q_2 at (1,3) and move to next row.
Step 9: Place Q_3 at (2,0) and move to next row.
Step 10: Place Q_4 at (3,2) and move to next row.

Possible Configuration of solution:

	0	1	2	3
0	x	Q_1	x	x
1	x	x	x	Q_2
2	Q_3	x	x	x
3	x	x	Q_4	x

22BCON1068

PAGE NO.: 17
DATE: / /

safe

• Section C : (3×11=33)

1. (a) Explain the fractional knapsack Problem and the concept of the Greedy Algorithm used to solve it. Discuss how the algorithm makes choices and its overall strategy.

i.e.,

(b) Consider a set of items with weights and values as follows:

Item	weight (w)	value (v)
1	2	10
2	3	5
3	5	15
4	7	7
5	1	6

3. Using the Greedy Algo. for the fractional knapsack Problem, determine the optimal selection of items to maximize the total value while not exceeding a knapsack capacity of 10 units. Should your step-by-step choices.

Ans (a) The fractional knapsack problem is also one of the techniques which are used to solve the knapsack problem. In fractional knapsack, the items are broken in order to maximize the profit. The problem in which we break the item is known as fractional knapsack problem. it can be solved using Greedy algorithm.

In Greedy algorithm, we calculate the ratio of profit/weight and accordingly, we will select the item. The item with the highest ratio would be selected first.

(b) (i)

Item	weight (w)	value (v)
1	2	10
2	3	5
3	5	15
4	7	7
5	1	6

Oct 6, 2024, 22:47

22BCON1068
Ankit Kumar
PAGE NO.: 18
DATE: / /

2.(a)

(b)

Knapsack capacity = 10 units

Step (ii) Value to weight ratio : (V/W)

Item	Weight (W)	Value (V)	V/W
1	2	10	5
2	3	5	1.67
3	5	15	3.0
4	7	7	1.0
5	1	6	6.0

Step(iii) - Sort the items in decending orders based on their value to weight ratio.

Item	Weight (W)	value (V)	V/W
5	1	6	6.0
1	2	10	5.0
3	5	15	3.0
2	3	5	1.67
4	7	7	1.0

Step (iv) → Start filling the items in knapsack capacity :

knapsack weight	item in knapsack	cost
10	0	0
(10-1) = 9	(5)	6
9-2 = 7	(5), (1)	6+10 = 16
7-5 = 2	(5), (1), (3)	16+15 = 31
2-2 = 0	(5), (1), (3)(2)	31+3.33 = 34.33

Now take function of item 2

The function we can take is 2/3 of its total value

i.e, $\frac{2}{3} \times 5 \approx 3.33$

Total maximum value = 6+10+15+3.33 = 34.33

Oct 6, 2024, 22:47

2.(a) Explain the 0/1 knapsack problem, detailing the problem statement, its significance, and the challenges associated with solving it.

(b) You are given a set of items, each with a weight (w) and a value (v), as follows:

Item	Weight (w)	Value (v)
1	2	10
2	3	5
3	5	15
4	7	7
5	1	6

Your Goal is to determine the optimal selection of items to maximize the total value while not exceeding a knapsack capacity of 10 units. You need to use the 0/1 knapsack problem approach to solve this.

Sol (a) We are given N items where each item has some weight (wi) and value (vi) associated with it. We are also given a bag with capacity W. The target is to put the items into the bag such that the sum of values associated with them is the maximum possible. Note that here we can either put an item completely into the bag or cannot put it at all. The problem can be expressed as:

Maximize $\sum_{i=1}^{N} Vixi$ subject to $\sum_{i=1}^{N} Wixi \leq W$ and

$$Xi \in \{0, 1\}$$

(b) $Wi = \{2, 3, 5, 7, 1\}$

$Vi = \{10, 5, 15, 7, 6\}$

Sol?

Using $m[i,w] = \max[m[i-1,w], m(i-1, w-w_i)] + p[i])$

		W→											
		i↓	0	1	2	3	4	5	6	7	8	9	10
Pi	Wi	0	0	0	0	0	0	0	0	0	0	0	0
6	1	1	0	6	6	6	6	6	6	6	6	6	6
10	2	2	0	6	10	16	16	16	16	16	16	16	16
5	3	3	0	6	10	16	16	16←	21	21	21	21	21
15	5	4	0	6	10	16	16	16←	21	25	31	31	31←
7	7	5	0	6	10	16	16	16	21	25	31	31	31←

$x_i = \{1, 0, 1, 0, 1\}$

Selected items :

Wi	Pi (value)
2	10
5	15
1	6

3. Given a seaquence of matrics of dimensions $A1, A2, A3 \dots An$, where the dimensions of the ith matrix are $Ai-1 \times Ai$ for $i = 1$ to n, your task is to find the most efficient way to multiply these matrics, minimize the total number of multiplications.

problem details :

→ Define the dim^n. of the matrics in the seaquence $A1, A2, A3 \dots An$

→ Calculate and explain the possible no of way to Parenthesize the

→ Calculate and Justify the cost of multiplication for each possible Parenthesization.

→ Determine the most efficient way to parenthesize the matrices to minimize the total no. of multiplications.

→ Calculate the minimum no. of multiplication required.

→ Provide a step-by-step explanation of your approach and calculations.

Oct 6, 2024, 22:48

Sol? (i) Define the dimensions: Let the dimensions of the matrics be $A_1, A_2, A_3 \dots A_n$ with matrix A_i having dimensions $P_{i-1} \times P_i$. This means we are multiplying matrices with dimensions $P_0 \times P_1$, $P_1 \times P_2 \dots P_{n-1} \times P_n$

(ii) $$C_n = \frac{(2n)!}{(n+1)!\,(n)!}$$

$$C_3 = \frac{6!}{4!\,3!} = 5 \text{ ways}$$

(iii) The cost of mulplying 2 matrices, A of size $P \times q$ and B of size $q \times n$ is

A $p \times q$ and B $q \times r$

Cost = $p \times q \times r$

(iv) Let $M[i][j]$ represent the min no. of scalars multiplication needed to compute the Product of matrices A_i to A_j for each pair of matrices $i \le j$

$$m[i][j] = \min [m[i][k] + m[k+1][j] + p_{i-1} * p_k * p_i)$$

(v) Calculate the min cost by efficient all possible ways to split the product. For example, if we have matrices with dimensions $10 \times 30 \times 5 \times 60$. The optimal split could be

$(A_1 \times A_2) \times (A_3 \times A_4)$.

(vi) Minimum no of multiplications: Using dp table, the min no. of multiplication can be computed efficiently.

for e.g; in the above scenario, the min no. of multiplications required is 4500.

Oct 6, 2024, 22:48